

AI4SWENG — Observability Integration Contract

Audience: All KIO Consortium Teams (KIO2–KIO13) **Issuer:** Central Platform Team (KIO1) **Status:** Normative

Page 1 — Architecture

High-Level: Telemetry Data Flow

Every KIO pushes two independent telemetry streams. KIOs never run their own collector, never expose a scrape/pull endpoint, and never connect directly to the central PostgreSQL instance. Dotted lines represent logical correlation mapping.

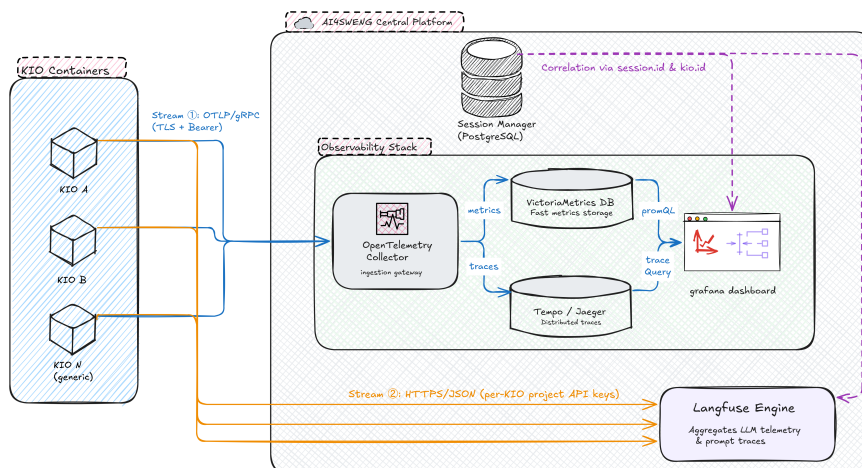


Figure 1: High-Level Architecture and Data Flow

Low-Level: In-Process Instrumentation Inside One KIO

This diagram shows how telemetry originates inside a KIO container, the SDK components involved, and the mandatory resource attributes that must be present on every telemetry signal as correlation keys back to the Session Manager.

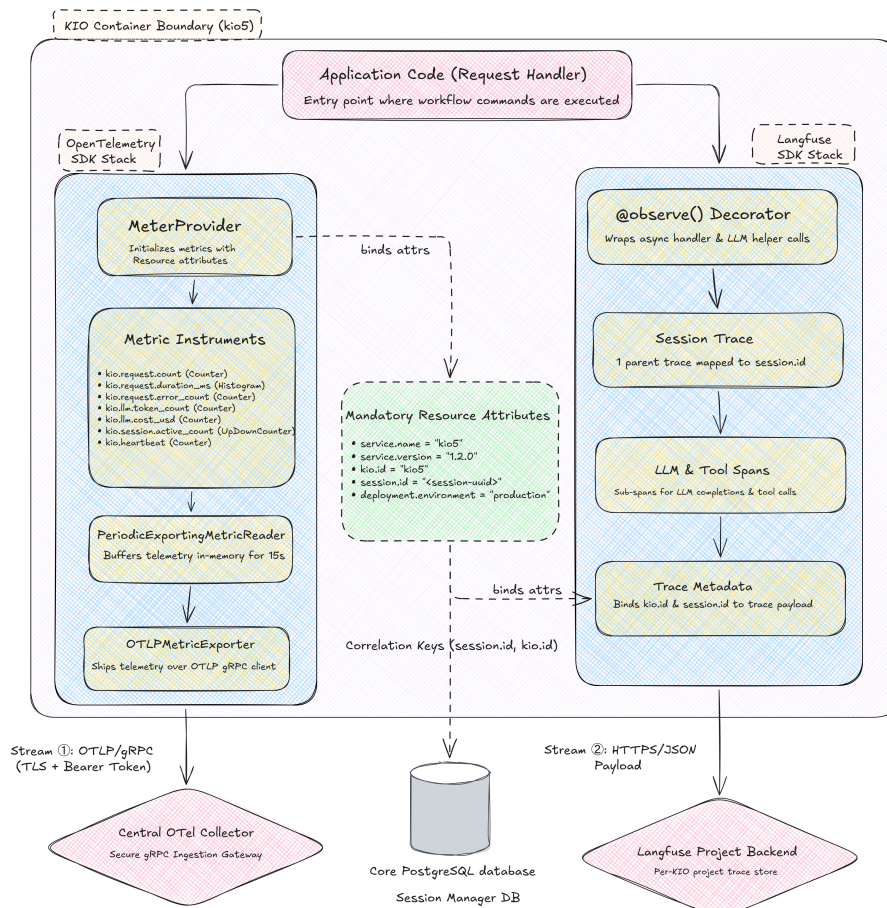


Figure 2: Detailed In-Process Instrumentation Diagram (Scaled Larger for Legibility)

Page 2 — Normative Contract

§1 Onboarding & Connectivity Requirements

This section defines the mandatory steps a KIO team must complete before their service is considered observability-onboarded. A KIO is not onboarded until a test metric and a test trace are both verified on the central platform.

1.1 Request Credentials

Contact the central platform team to receive: (1) **OTLP Bearer Token** to authorize OTLP exports to the central collector, and (2) **Langfuse Project Keys** (Public/Secret) for your KIO's dedicated Langfuse project.

1.2 Set Environment Variables

Inject the following variables into your KIO container environment:

```
1 OTEL_EXPORTER_OTLP_ENDPOINT="https://otel-collector.ai4sweng.io:4317"
2 OTEL_EXPORTER_OTLP_HEADERS="Authorization=Bearer <YOUR_OTLP_TOKEN>"
3 OTEL_RESOURCE_ATTRIBUTES="service.name=kio5,service.version=1.2.0,kio.id=kio5,deployment.environment=production"
4 LANGFUSE_PUBLIC_KEY="pk-lf-..."
5 LANGFUSE_SECRET_KEY="sk-lf-..."
6 LANGFUSE_HOST="https://langfuse.ai4sweng.io"
```

1.3 Install SDKs & 1.4 Attach Attributes

Install standard packages via pip: `opentelemetry-api`, `opentelemetry-sdk`, `opentelemetry-exporter-otlp-proto-grpc`, and `langfuse`. Ensure your OTEL MeterProvider loads resource attributes from `OTEL_RESOURCE_ATTRIBUTES`.

1.5 Verification Gate

Run your KIO and verify:

1. **Metrics:** The `kio.heartbeat` counter is visible in the central Grafana dashboard.
2. **Traces:** At least one trace is visible in your Langfuse project, tagged with `kio.id` and `session.id`.

§2 Telemetry Compliance Requirements

Every onboarded KIO must continuously export telemetry complying with the rules below.

2.1 Mandatory Metric Set

All metrics must carry the `kio.id` attribute.

Metric Name	Type	Unit	Labels	Description
<code>kio.request.count</code>	Counter	1	<code>kio.id</code> , <code>status</code>	Total requests processed (<code>status: ok/error</code>)
<code>kio.request.duration.ms</code>	Histogram	ms	<code>kio.id</code>	End-to-end latency distribution
<code>kio.request.error.count</code>	Counter	1	<code>kio.id</code> , <code>error.type</code>	Errors categorized by bounded types
<code>kio.llm.token.count</code>	Counter	tokens	<code>kio.id</code> , <code>direction</code>	LLM token usage (<code>direction: input/output</code>)
<code>kio.llm.cost.usd</code>	Counter	USD	<code>kio.id</code>	Total estimated LLM cost
<code>kio.session.active.count</code>	UpDownCounter	1	<code>kio.id</code>	Active in-flight sessions
<code>kio.heartbeat</code>	Counter	1	<code>kio.id</code>	Background heartbeat incremented every 60s

2.2 Langfuse Trace Requirements

- **Project Isolation:** Emitters must push only to their assigned project (no cross-consortium project sharing).
- **Session Linkage:** Open exactly one Langfuse trace per workflow session using the `session.id` as the trace's `session.id`.
- **Structure:** Each LLM call and tool call must be a sub-span nested inside the session trace.

2.3 Continuous Export & Heartbeat

Telemetry must be continuously exported. The `kio.heartbeat` metric must tick every 60 seconds. A KIO failing to export a heartbeat for more than 120 seconds will be marked as stale in the central registry.

Appendix — Implementation & Guidelines

Reference Implementation

This Python snippet registers a globally configured OpenTelemetry `MeterProvider` with periodic OTLP gRPC export, and wraps the request logic with Langfuse tracing.

```

1 import os, time, asyncio
2 from opentelemetry import metrics
3 from opentelemetry.sdk.metrics import MeterProvider
4 from opentelemetry.sdk.metrics.export import PeriodicExportingMetricReader
5 from opentelemetry.exporter.otlp.proto.grpc.metric_exporter import OTLPMetricExporter
6 from opentelemetry.sdk.resources import Resource
7 from langfuse.decorators import observe, langfuse_context
8
9 # 1. OpenTelemetry Setup
10 resource = Resource.create({
11     k: v for attr in os.environ.get("OTEL_RESOURCE_ATTRIBUTES", "").split(",")
12     for k, v in [attr.split("=")] if "=" in attr
13 })
14 exporter = OTLPMetricExporter(insecure=False)
15 reader = PeriodicExportingMetricReader(exporter, export_interval_millis=15000)
16 metrics.set_meter_provider(MeterProvider(resource=resource, metric_readers=[reader]))
17 meter = metrics.get_meter("kio.instrumentation")
18
19 # 2. Mandatory Instruments
20 request_counter = meter.create_counter("kio.request.count", unit="1")
21 request_duration = meter.create_histogram("kio.request.duration_ms", unit="ms")
22 error_counter = meter.create_counter("kio.request.error_count", unit="1")
23 llm_token_counter = meter.create_counter("kio.llm.token_count", unit="tokens")
24 llm_cost_counter = meter.create_counter("kio.llm.cost_usd", unit="USD")
25 active_sessions = meter.create_up_down_counter("kio.session.active_count", unit="1")
26 heartbeat_counter = meter.create_counter("kio.heartbeat", unit="1")
27 KIO_ID = os.environ.get("KIO_ID", "kioN")
28
29 # Background heartbeat task
30 async def _heartbeat_loop():
31     while True:
32         heartbeat_counter.add(1, {"kio.id": KIO_ID})
33         await asyncio.sleep(60)
34
35 # Request handler metrics wrapper
36 async def handler(envelope):
37     session_id = envelope.get("session_id", "unknown")
38     labels = {"kio.id": KIO_ID}
39     active_sessions.add(1, labels)
40     start = time.monotonic()
41     try:
42         result = await _do_llm_work(session_id, envelope)
43         request_counter.add(1, {**labels, "status": "ok"})
44         request_duration.record((time.monotonic() - start) * 1000, labels)
45         return result
46     except Exception as exc:
47         request_counter.add(1, {**labels, "status": "error"})
48         request_duration.record((time.monotonic() - start) * 1000, labels)
49         error_counter.add(1, {**labels, "error_type": "internal"})
50         raise
51     finally:
52         active_sessions.add(-1, labels)
53
54 @observe(name="llm_call")
55 async def _do_llm_work(session_id: str, envelope: dict):
56     langfuse_context.update_current_trace(
57         session_id=session_id,
58         metadata={"kio.id": KIO_ID, "session.id": session_id},
59     )
60     # LLM completion call goes here...

```

Guidelines

#	Rule Definition
G1	Dot-style stable names: Use the pattern <code>kio.<domain>.<metric></code> . Names are permanent once released.
G2	Always include correlation keys: Every metric and trace/span must include <code>kio.id</code> and <code>session.id</code> .
G3	No high-cardinality labels: Do not use unique IDs, URLs, or emails as label values. Use bounded enums.
G4	No secrets or PII: Never leak API keys, user passwords, or email addresses in traces or metric labels.
G5	Set service version: Update the <code>service.version</code> attribute with semantic versioning on every release.
G6	Mandatory metrics governance: Central platform team announces and standardizes mandatory metrics.
G7	Optional metrics are self-service: Consortium teams can define custom metrics matching rules G1–G5.